Keras課堂實驗(一)

三種鳶尾花

高怡宣
中央大學 通識教育中心

Iris flower data set 三種鳶尾花

The multivariate data set introduced by the British statistician and biologist Ronald Fisher in 1936. The data set consists of 50 samples from each of three species of Iris (Iris setosa, Iris virginica and Iris versicolor). Four features were measured from each sample: the length and the width of the sepals (萼片) and petals (花瓣). (from Wikipedia)

Iris setosa. CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=170298

Iris versicolor, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=248095

Iris virginica,

By Frank Mayfield , CC BY-SA 2.0, https://commons.wikimedia.org/w/index.php?curid=9805580

Data: Classify iris plants into three species
https://www.kaggle.com/uciml/iris/data

Code: Sequential ANN and Visualization
https://www.kaggle.com/pratik2901/sequential-ann-and-visualization

python

ANACONDA

Keras: The Python Deep Learning library

Keras

jupyter

TensorFlow

kaggle

kaggle | Search kaggle | Competitions | Datasets | Kernels | Discussion | Learn | ••• | Sign In

Reviewed Dataset

589

Iris Species

Classify iris plants into three species in this classic dataset

三種鳶尾花

UCI ML UCI Machine Learning • last updated 2 years ago

Data | Overview | Kernels | Discussion | Activity | Download (4 KB) | New Kernel

Data | API | ? | Download All

Data Sources

- Iris.csv 150 x 6
- database.sqlite
  - Iris 150 x 6

About this file

SQLite database containing the same data as Iris.csv

Columns

- # Id Unique ID
- # SepalLengthCm Length of the sepal (in cm)

https://www.kaggle.com/uciml/iris/data

kaggle Search kaggle Competitions Datasets Kernels Dis

Prateek

Sequential ANN And Visualization

last run 10 months ago · IPython Notebook HTML · 186 views
using data from Iris Species · Public

Notebook Code Data (1) Comments (0) Log Versions (4) Forks (1)

Notebook

https://www.kaggle.com/pratik2901/sequential-ann-and-visualization

File Edit View Insert Cell Kernel Widgets Help

Run Code

Iris

sepal: 萼片, petal: 花瓣

data from: https://www.kaggle.com/uciml/iris/data

code modified from: https://www.kaggle.com/pratik2901/sequential-ann-and-visualization

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import np_utils
from sklearn.preprocessing import LabelEncoder
from keras import optimizers
```

```
C:\Users\USER\Anaconda3\lib\site-packages\h5py\__i
argument of issubdtype from `float` to `np.floatin
np.float64 == np.dtype(float).type`.
  from ._conv import register_converters as _regis
Using TensorFlow backend.
```

In [2]:
```
seed = 3
np.random.seed(seed)
```

In [3]:
```
iris = pd.read_csv("../input/Iris.csv")
```

pandas.read_csv

pandas.**read_csv**(*filepath_or_buffer, sep=', ', delimiter=None, header='infer', names=None, index_col=None, usecols=None, squeeze=False, prefix=None, mangle_dupe_cols=True, dtype=None, engine=None, converters=None, true_values=None, false_values=None, skipinitialspace=False, skiprows=None, nrows=None, na_values=None, keep_default_na=True, na_filter=True, verbose=False, skip_blank_lines=True, parse_dates=False, infer_datetime_format=False, keep_date_col=False, date_parser=None, dayfirst=False, iterator=False, chunksize=None, compression='infer', thousands=None, decimal=b'.', lineterminator=None, quotechar='"', quoting=0, escapechar=None, comment=None, encoding=None, dialect=None, tupleize_cols=None, error_bad_lines=True, warn_bad_lines=True, skipfooter=0, doublequote=True, delim_whitespace=False, low_memory=True, memory_map=False, float_precision=None*) [source]

Read CSV (comma-separated) file into DataFrame

https://pandas.pydata.org/pandas-docs/stable/generated/pandas.read_csv.html#pandas.read_csv

pandas.DataFrame

class pandas.**DataFrame**(*data=None*, *index=None*, *columns=None*, *dtype=None*, *copy=False*) [source]

Two-dimensional size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns). Arithmetic operations align on both row and column labels. Can be thought of as a dict-like container for Series objects. The primary pandas data structure.

Pandas.DataFrame(data, index, column, dtype, copy)

https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.html

Parameters:

data : *numpy ndarray (structured or homogeneous), dict, or DataFrame*

Dict can contain Series, arrays, constants, or list-like objects *Changed in version 0.23.0:* If data is a dict, argument order is maintained for Python 3.6 and later.

index : *Index or array-like*

Index to use for resulting frame. Will default to RangeIndex if no indexing information part of input data and no index provided

columns : *Index or array-like*

Column labels to use for resulting frame. Will default to RangeIndex (0, 1, 2, …, n) if no column labels are provided

dtype : *dtype, default None*

Data type to force. Only a single dtype is allowed. If None, infer

copy : *boolean, default False*

Copy data from inputs. Only affects DataFrame / 2d ndarray input

https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.html

Examples

Constructing DataFrame from a dictionary.

```
>>> d = {'col1': [1, 2], 'col2': [3, 4]}
>>> df = pd.DataFrame(data=d)
>>> df
   col1  col2
0     1     3
1     2     4
```

https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.html

Constructing DataFrame from numpy ndarray:

```
>>> df2 = pd.DataFrame(np.random.randint(low=0, high=10, size=(5, 5)),
...                    columns=['a', 'b', 'c', 'd', 'e'])
>>> df2
   a  b  c  d  e
0  2  8  8  3  4
1  4  2  9  0  9
2  1  0  7  8  0
3  5  1  7  1  3
4  6  0  2  4  2
```

https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.html

Attributes (屬性)

T	Transpose index and columns.
at	Access a single value for a row/column label pair.
axes	Return a list representing the axes of the DataFrame.
blocks	(DEPRECATED) Internal property, property synonym for as_blocks()
columns	The column labels of the DataFrame.
dtypes	Return the dtypes in the DataFrame.
empty	Indicator whether DataFrame is empty.
ftypes	Return the ftypes (indication of sparse/dense and dtype) in DataFrame.
iat	Access a single value for a row/column pair by integer position.
iloc	Purely integer-location based indexing for selection by position.
index	The index (row labels) of the DataFrame.
ix	A primarily label-location based indexer, with integer position fallback.
loc	Access a group of rows and columns by label(s) or a boolean array.
ndim	Return an int representing the number of axes / array dimensions.
shape	Return a tuple representing the dimensionality of the DataFrame.
size	Return an int representing the number of elements in this object.
style	Property returning a Styler object containing methods for building a styled HTML representation fo the DataFrame.
values	Return a Numpy representation of the DataFrame.

https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.html

Methods (方法)

abs()

add(other[, axis, level, fill_valu

add_prefix(prefix)

add_suffix(suffix)

agg(func[, axis])

aggregate(func[, axis])

align(other[, join, axis, level,

all([axis, bool_only, skipna, l

any([axis, bool_only, skipna, l

append(other[, ignore_index, .

apply(func[, axis, broadcast, r

applymap(func)

cumprod([axis, skipna])

cumsum([axis, skipna])

describe([percentiles, include, exclude])

diff([periods, axis])

div(other[, axis, level, fill_value])

divide(other[, axis, level, fill_value])

dot(other)

drop([labels, axis, index, columns, level, …])

drop_duplicates([subset, keep, inplace])

dropna([axis, how, thresh, subset, inplace])

duplicated([subset, keep])

get_ftype_counts()

get_value(index, col[, takeable])

get_values()

groupby([by, axis, level, as_index, sort, …])

gt(other[, axis, level])

head([n])

hist([column, by, grid, xlabelsize, xrot, …])

idxmax([axis, skipna])

idxmin([axis, skipna])

infer_objects()

info([verbose, buf, max_cols, memory_usage, …])

insert(loc, column, value[, allow_duplicates])

interpolate([method, axis, limit, inplace, …])

https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.html

In [4]:
```
iris.head()  # .head gives the first 5 rows
```

Out[4]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

In [5]:
```
col = iris.columns        # .columns gives columns names in data
print(col)
```

```
Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')
```

```
In [3]: iris = pd.read_csv("../input/Iris.csv")
```

```
In [ ]: iris.
```

```
In [4]: iris.head()  # .head gives the first 5 rows
```

按 Tab, 找DataFrame的用法

```
argument of issubdtype from `float` to `np.floating` is deprecated.
In [2]:                                  float).type`.
                                         ister_converters as _register_converters
                                         l.

In [3]:                                  nput/Iris.csv")

In [ ]: iris.
```

```
iris.Id
iris.PetalLengthCm
iris.PetalWidthCm
iris.SepalLengthCm
iris.SepalWidthCm
iris.Species
iris.T
iris.abs
iris.add
iris.add_prefix
```

```
In [4]: iris.head()  # .head gives the first 5 rows
```

Out[4]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

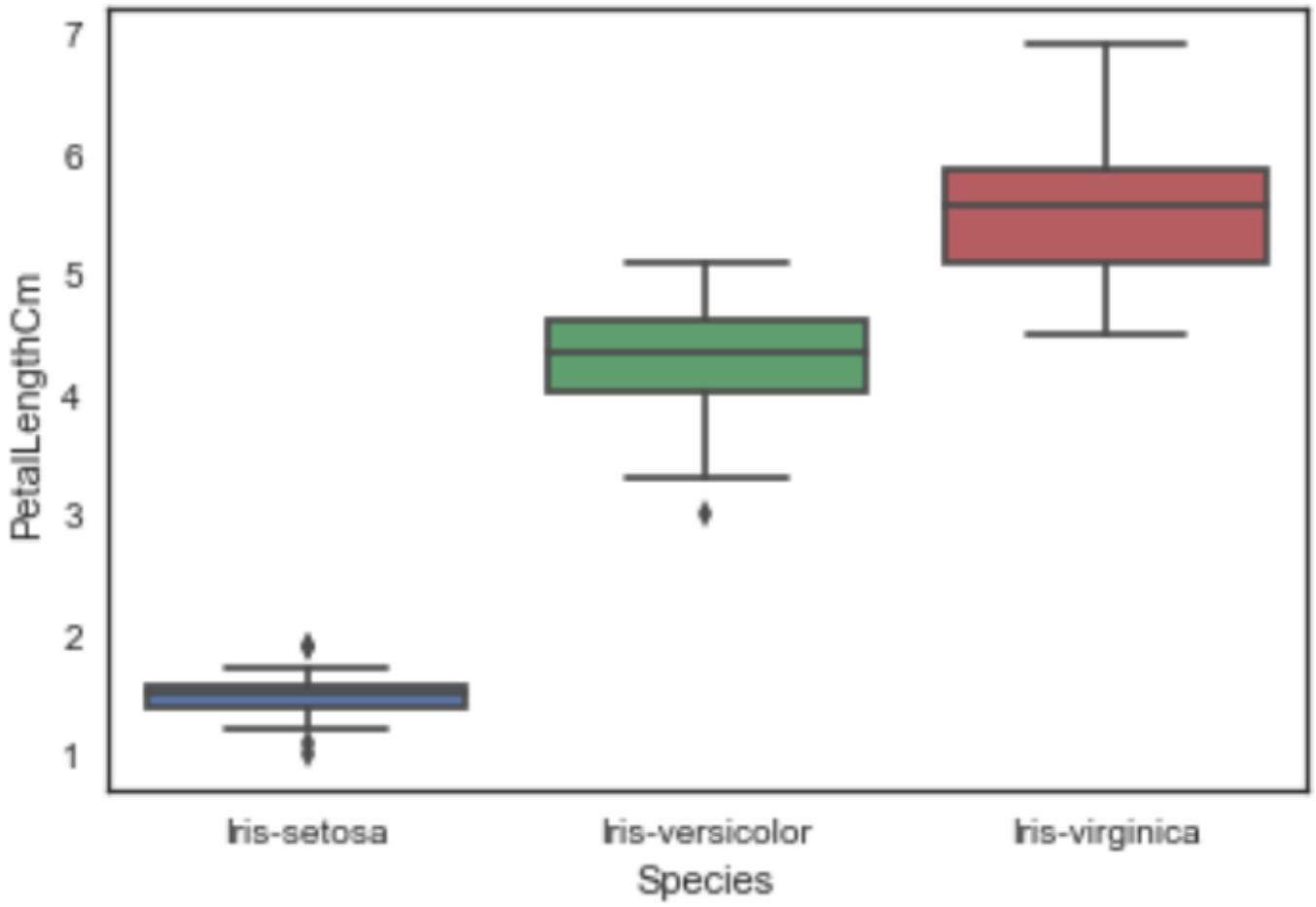
Visualization

In [6]:
```
iris["Species"].value_counts()
```

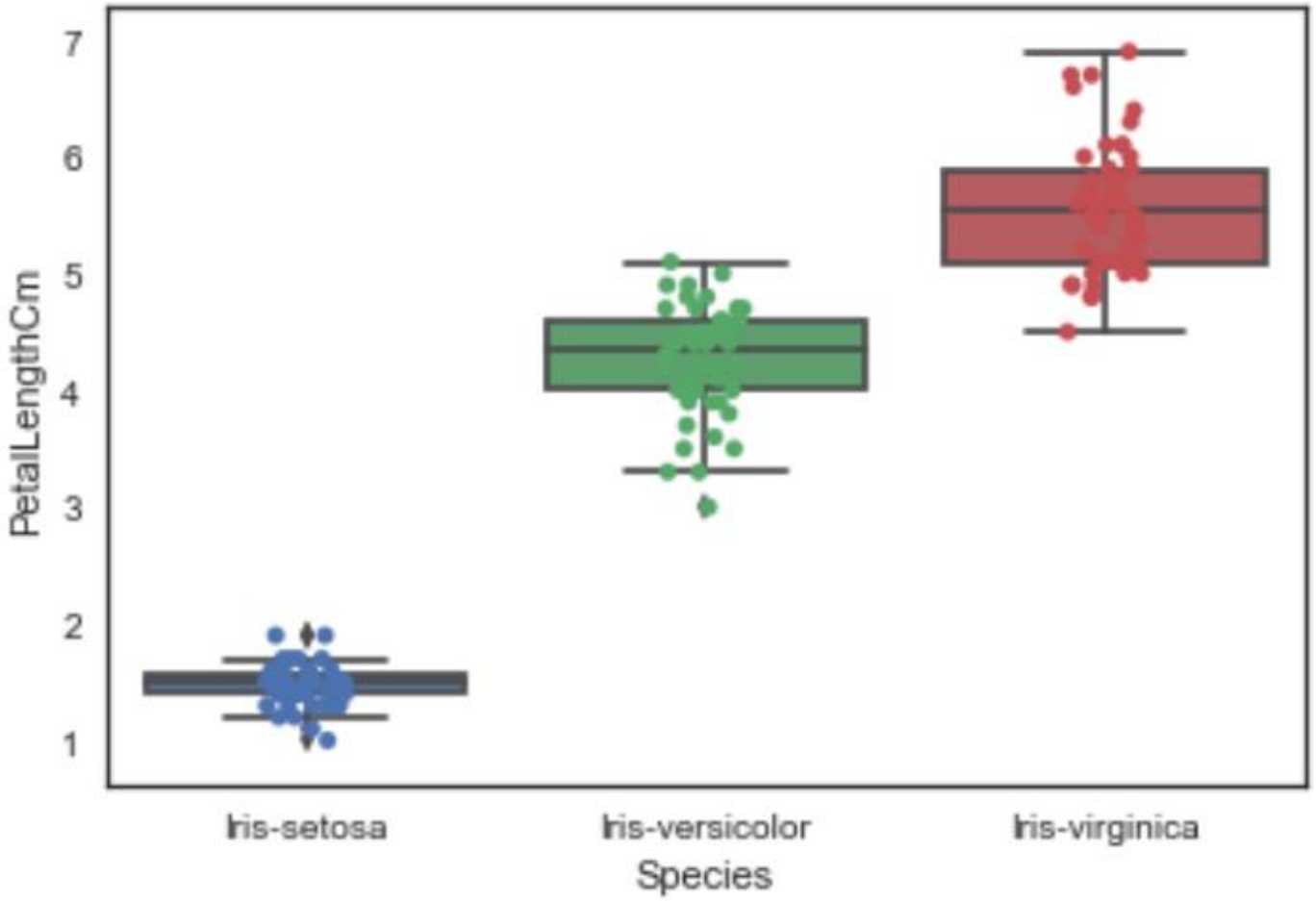
Out[6]:
```
Iris-versicolor    50
Iris-setosa        50
Iris-virginica     50
Name: Species, dtype: int64
```

In [7]:
```
sns.set(style='white', color_codes=True)
sns.boxplot(x="Species", y="PetalLengthCm", data=iris)
```

Out[7]:
```
<matplotlib.axes._subplots.AxesSubplot at 0xea76780>
```



```
In [8]: ax = sns.boxplot(x="Species", y="PetalLengthCm", data=iris)
        ax = sns.stripplot(x="Species", y="PetalLengthCm", data=iris,
```



seaborn.boxplot

`seaborn.boxplot` (*x=None, y=None, hue=None, data=None, order=None, hue_order=None, orient=None, color=None, palette=None, saturation=0.75, width=0.8, dodge=True, fliersize=5, linewidth=None, whis=1.5, notch=False, ax=None, **kwargs*)

Draw a box plot to show distributions with respect to categories.

A box plot (or box-and-whisker plot) shows the distribution of quantitative data in a way that

https://seaborn.pydata.org/generated/seaborn.boxplot.html

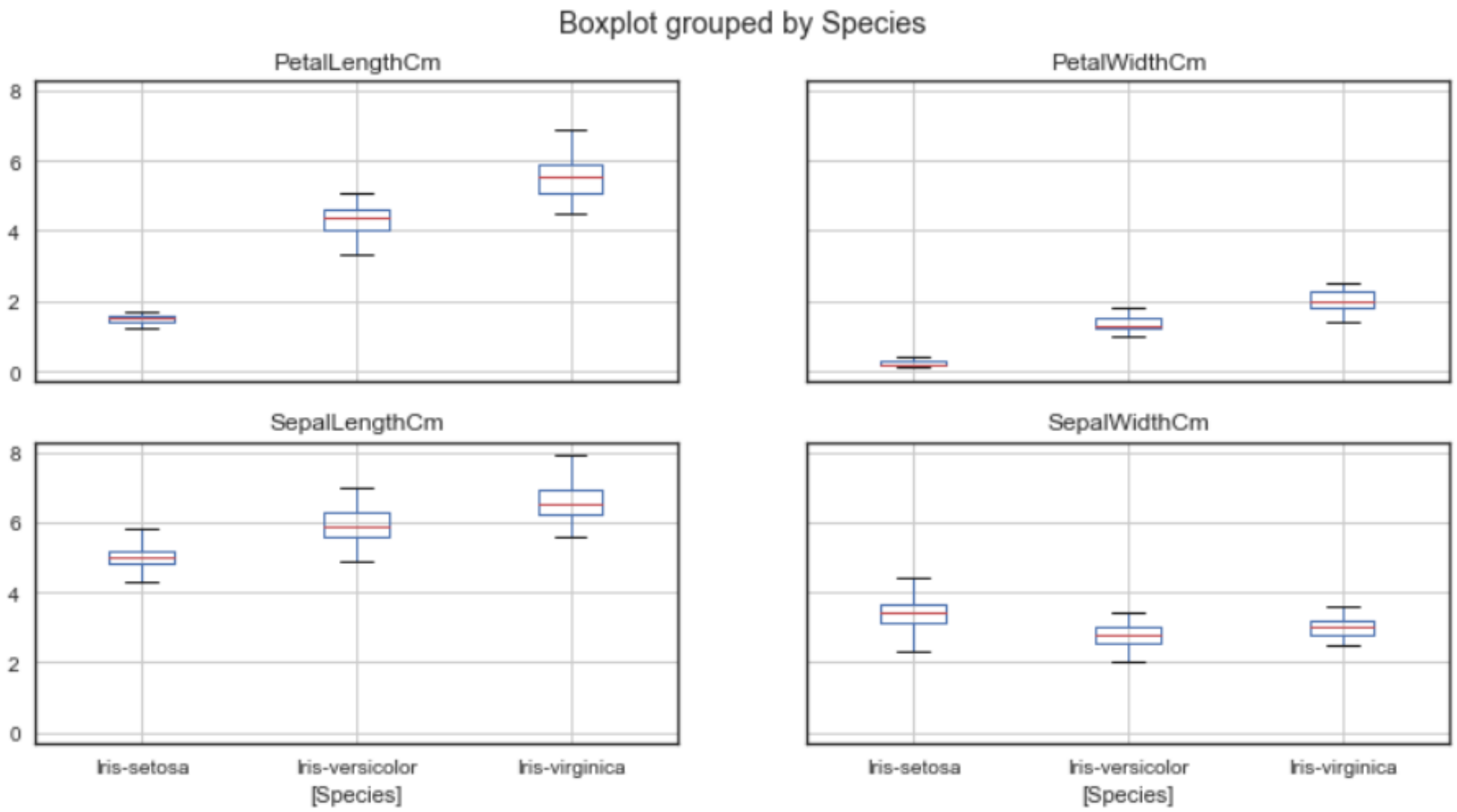
seaborn.stripplot

`seaborn.stripplot` (*x=None, y=None, hue=None, data=None, order=None, hue_order=None, jitter=True, dodge=False, orient=None, color=None, palette=None, size=5, edgecolor='gray', linewidth=0, ax=None, **kwargs*)

Draw a scatterplot where one variable is categorical.

https://seaborn.pydata.org/generated/seaborn.stripplot.html

```
In [9]: iris.drop("Id", axis=1).boxplot(by="Species", figsize=(12, 6))
```



pandas.DataFrame.drop

`DataFrame.drop`(*labels=None, axis=0, index=None, columns=None, level=None, inplace=False, errors='raise'*) [source]

Drop specified labels from rows or columns.

Remove rows or columns by specifying label names and corresponding axis, or by specifying directly index or column names. When using a multi-index, labels on different levels can be removed by specifying the level.

Parameters:

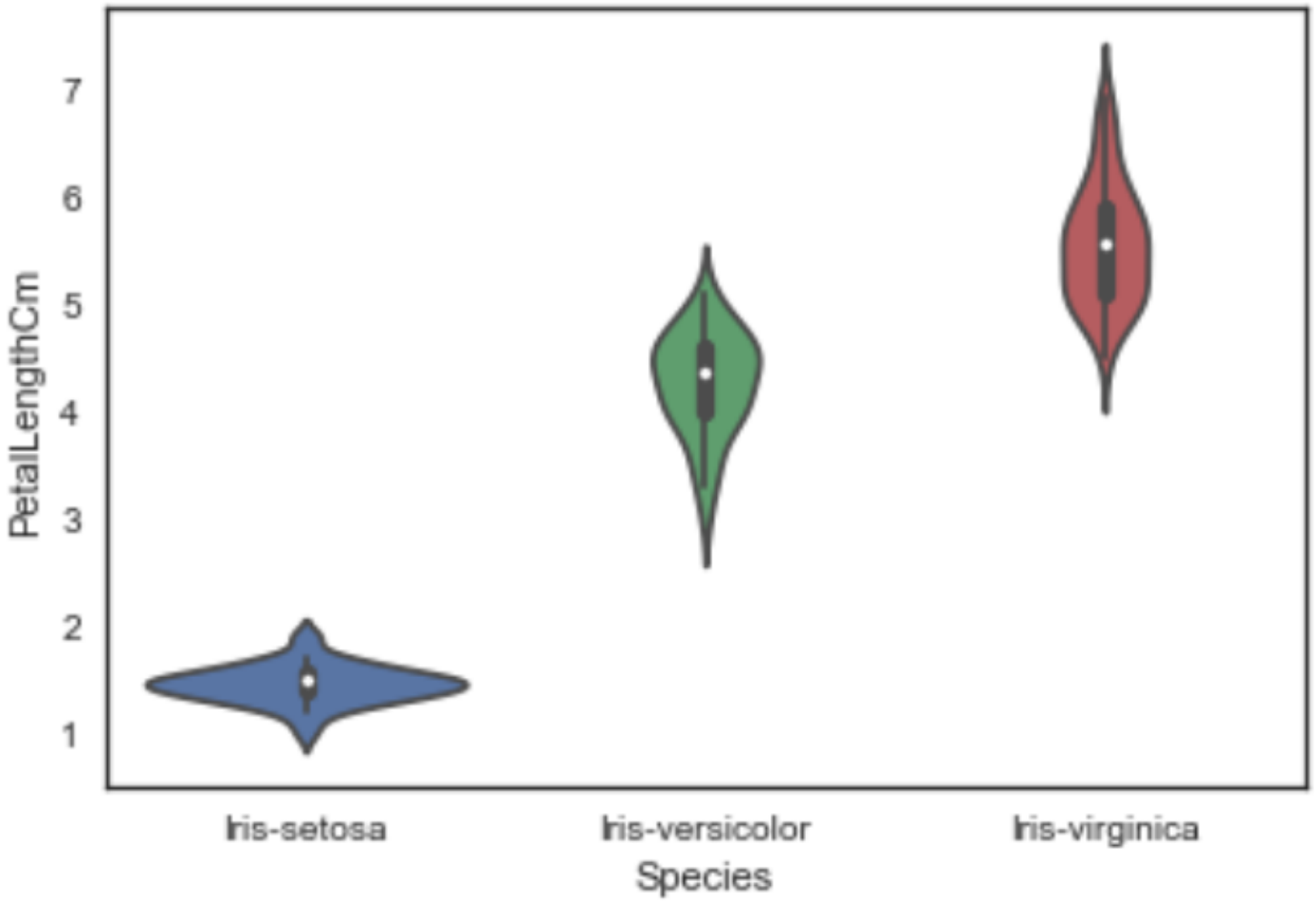
labels : *single label or list-like*
Index or column labels to drop.

axis : *{0 or 'index', 1 or 'columns'}, default 0*
Whether to drop labels from the index (0 or 'index') or columns (1 or 'columns').

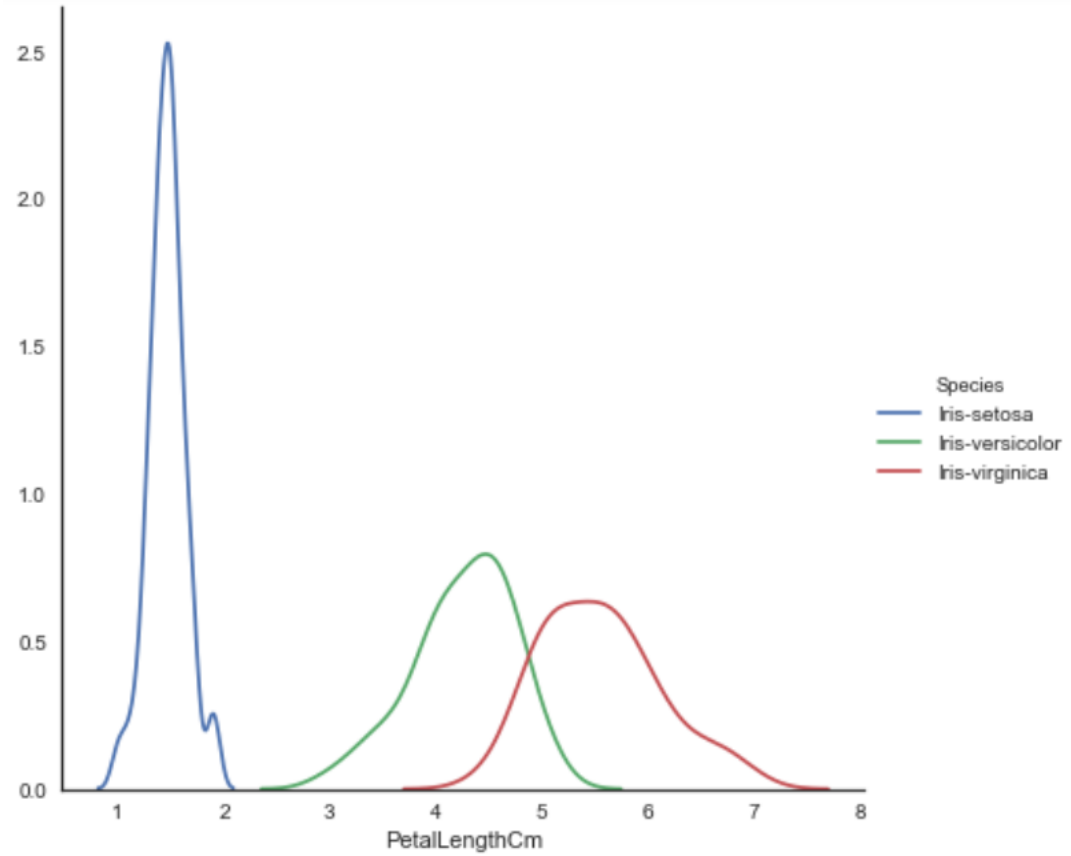
index, columns : *single label or list-like*
Alternative to specifying axis (`labels, axis=1` is equivalent to `columns=labels`).
New in version 0.21.0.

```
In [10]: sns.violinplot(x="Species", y="PetalLengthCm", data=iris, size=6)

Out[10]: <matplotlib.axes._subplots.AxesSubplot at 0xf326b38>
```



In [11]:
```
sns.FacetGrid(iris, hue="Species", size=6) \
    .map(sns.kdeplot, "PetalLengthCm") \
    .add_legend()
```



seaborn.FacetGrid

class `seaborn.FacetGrid` (*data, row=None, col=None, hue=None, col_wrap=*
height=3, aspect=1, palette=None, row_order=None, col_order=None, hue_orde
legend_out=True, despine=True, margin_titles=False, xlim=None, ylim=None, s
size=None)

Multi-plot grid for plotting conditional relationships.

https://seaborn.pydata.org/generated/seaborn.FacetGrid.html

seaborn.kdeplot

`seaborn.kdeplot` (*data, data2=None, shade=False, vertical=False, kernel=*

gridsize=100, cut=3, clip=None, legend=True, cumulative=False, shade_lowe

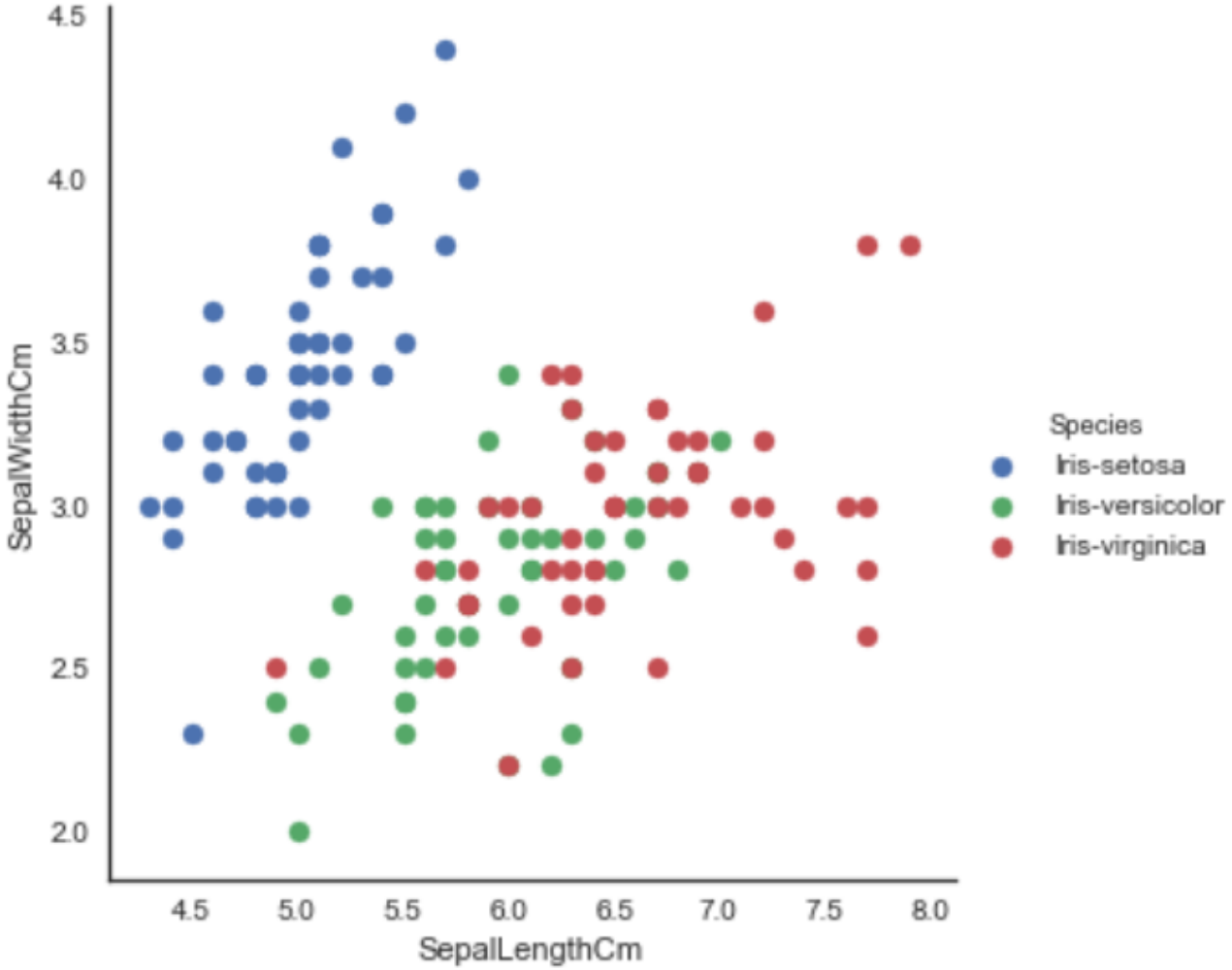
*cbar=False, cbar_ax=None, cbar_kws=None, ax=None, **kwargs*)

Fit and plot a univariate or bivariate kernel density estimate.

https://seaborn.pydata.org/generated/seaborn.kdeplot.html

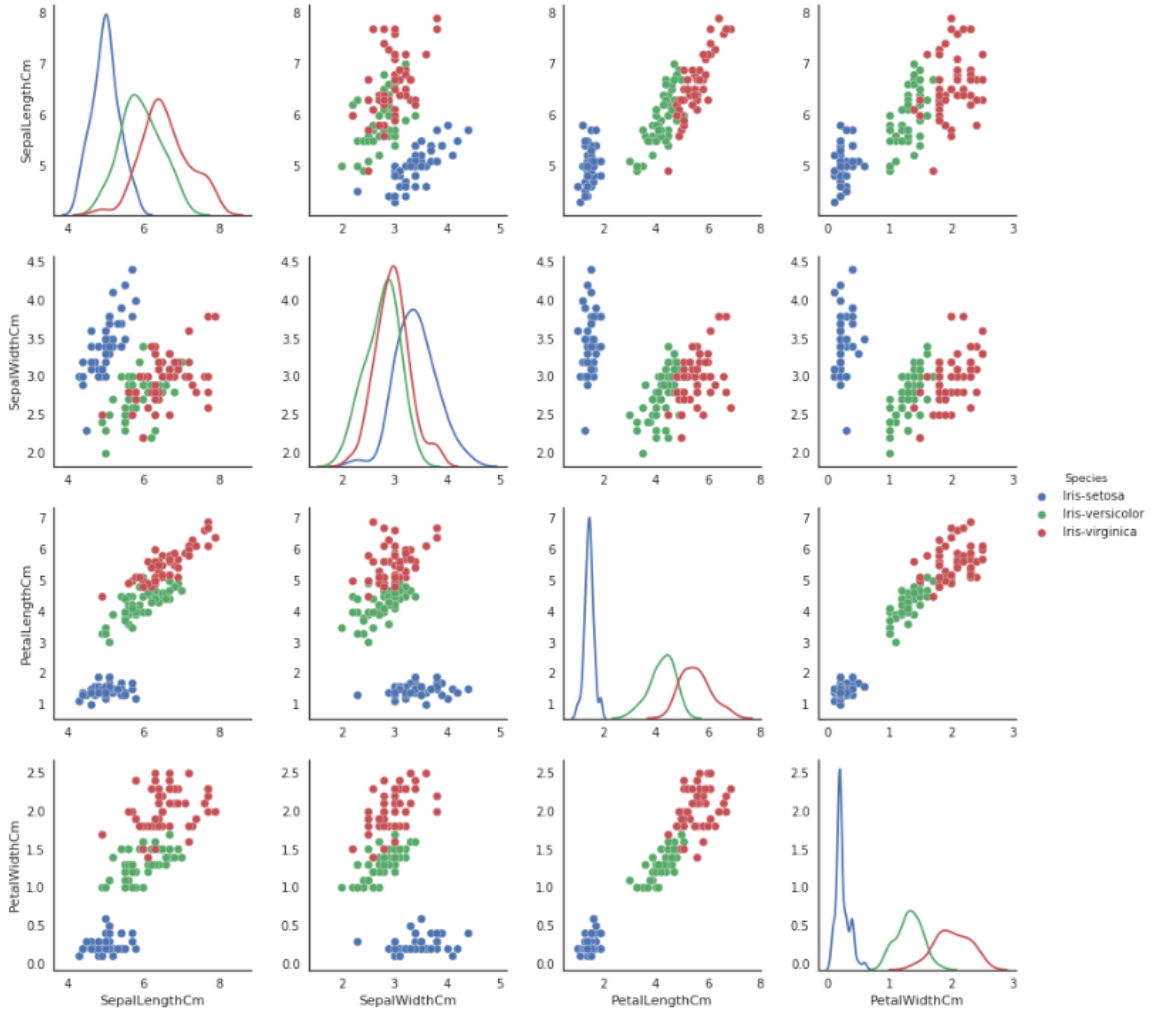
In [13]:
```
sns.FacetGrid(iris, hue="Species", size=5) \
    .map(plt.scatter, "SepalLengthCm", "SepalWidthCm") \
    .add_legend()
```

Out[13]: <seaborn.axisgrid.FacetGrid at 0x23c03e8a668>



```
In [14]: sns.pairplot(iris.drop("Id", axis=1), hue="Species", size=3, diag_kind="kde")
```

```
Out[14]: <seaborn.axisgrid.PairGrid at 0x23c03f0ce80>
```



Label Encoding

In [15]:
```
dataset = iris.values
X = dataset[:,1:5].astype(float)
print(X[0:5])
#print(X)
```

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
```

Out[4]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

Visualization

In [6]:
```
iris["Species"].value_counts()
```

Out[6]:
```
Iris-versicolor    50
Iris-setosa        50
Iris-virginica     50
Name: Species, dtype: int64
```

```python
# convert text into vectors
# method #1
Y = dataset[:,5]
print(Y)
```

```
['Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor'
 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor'
 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor'
```

```
'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor'
'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor'
'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor'
'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor'
'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor'
'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor'
'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor'
'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica' 'Iris-virginica'
'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica']
```

```
In [17]: Y_df = pd.DataFrame(Y)
         print(Y_df)
```

```
                    0
0        Iris-setosa
1        Iris-setosa
2        Iris-setosa

147  Iris-virginica
148  Iris-virginica
149  Iris-virginica

[150 rows x 1 columns]
```

One-hot

From Wikipedia, the free encyclopedia

In digital circuits, **one-hot** is a group of bits among which the legal combinations of values are only those with a single high (1) bit and all the others low (0).[1] A similar implementation in which all bits are '1' except one '0' is sometimes called **one-cold**.

Binary	Gray code	One-hot
000	000	00000001
001	001	00000010
010	011	00000100
011	010	00001000
100	110	00010000
101	111	00100000
110	101	01000000
111	100	10000000

One-hot encoding

ID	Gender
1	Male
2	Female
3	Not Specified
4	Not Specified
5	Female

ID	Male	Female	Not Specified
1	1	0	0
2	0	1	0
3	0	0	1
4	0	0	1
5	0	1	0

One-hot encoding 示意圖

https://medium.com/@yehjames/%E8%B3%87%E6%96%99%E5%88%86%E6%9E%90-%E6%A9%9F%E5%99%A8%E5%AD%B8%E7%BF%92-%E7%AC%AC2-4%E8%AC%9B-%E8%B3%87%E6%96%99%E5%89%8D%E8%99%95%E7%90%86-missing-data-one-hot-encoding-feature-scaling-3b70a7839b4a

In [18]:
```
Y_onehot = pd.get_dummies(Y_df)
print(Y_onehot.shape)
print(Y_onehot)
```

```
(150, 3)
     0_Iris-setosa  0_Iris-versicolor  0_Iris-virginica
0                1                  0                 0
1                1                  0                 0
2                1                  0                 0
3                1                  0                 0
4                1                  0                 0
..             ...                ...               ...
145              0                  0                 1
146              0                  0                 1
147              0                  0                 1
148              0                  0                 1
149              0                  0                 1

[150 rows x 3 columns]
```

In [19]:
```
# convert text into vectors
# method #2
Y = dataset[:,5]
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)
print(Y[0:5])
print(encoded_Y)
```

```
['Iris-setosa' 'Iris-setosa' 'Iris-setosa'
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1
```

In [20]:
```
answer_Y = np_utils.to_categorical(encoded_Y)
print(answer_Y.shape)
print(answer_Y[0:5])
```

```
(150, 3)
[[1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]]
```

Getting started with the Keras Sequential model

The `Sequential` model is a linear stack of layers.

You can create a `Sequential` model by passing a list of layer instances to the constructor:

```
from keras.models import Sequential
from keras.layers import Dense, Activation

model = Sequential([
    Dense(32, input_shape=(784,)),
    Activation('relu'),
    Dense(10),
    Activation('softmax'),
])
```

You can also simply add layers via the `.add()` method:

```
model = Sequential()
model.add(Dense(32, input_dim=784))
model.add(Activation('relu'))
```

https://keras.io/getting-started/sequential-model-guide/

*Sequential Based ANN *

```
def baseline_model():
    model = Sequential()
    model.add(Dense(8, input_dim=4, activation='relu'))
    model.add(Dense(3, activation='softmax'))
    model.compile(loss='categorical_crossentropy', optimizer='Adam',
    return model
```

Hidden layer: 8 neurons and 4 inputs
Output layer: 3 outputs

*Sequential Based ANN *

```
def baseline_model():
    model = Sequential()
    model.add(Dense(8, input_dim=4, activation='relu'))
    model.add(Dense(3, activation='softmax'))
    model.compile(loss='categorical_crossentropy', optimizer='Adam',
    return model
```

hidden-layer activation = ‘relu’

output-layer activation = ‘softmax’

loss = ‘categorical_crossentropy’

optimizer = ‘adam’

```python
model = baseline_model()
model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_1 (Dense)              (None, 8)                 40
_________________________________________________________________
dense_2 (Dense)              (None, 3)                 27
=================================================================
Total params: 67
Trainable params: 67
Non-trainable params: 0
_________________________________________________________________
```

(4w+1b) x (8 neurons) = 40

(8w+1b) x (3 neurons) = 27

```
history = model.fit(X,answer_Y,epochs=90,batch_size=5)
```

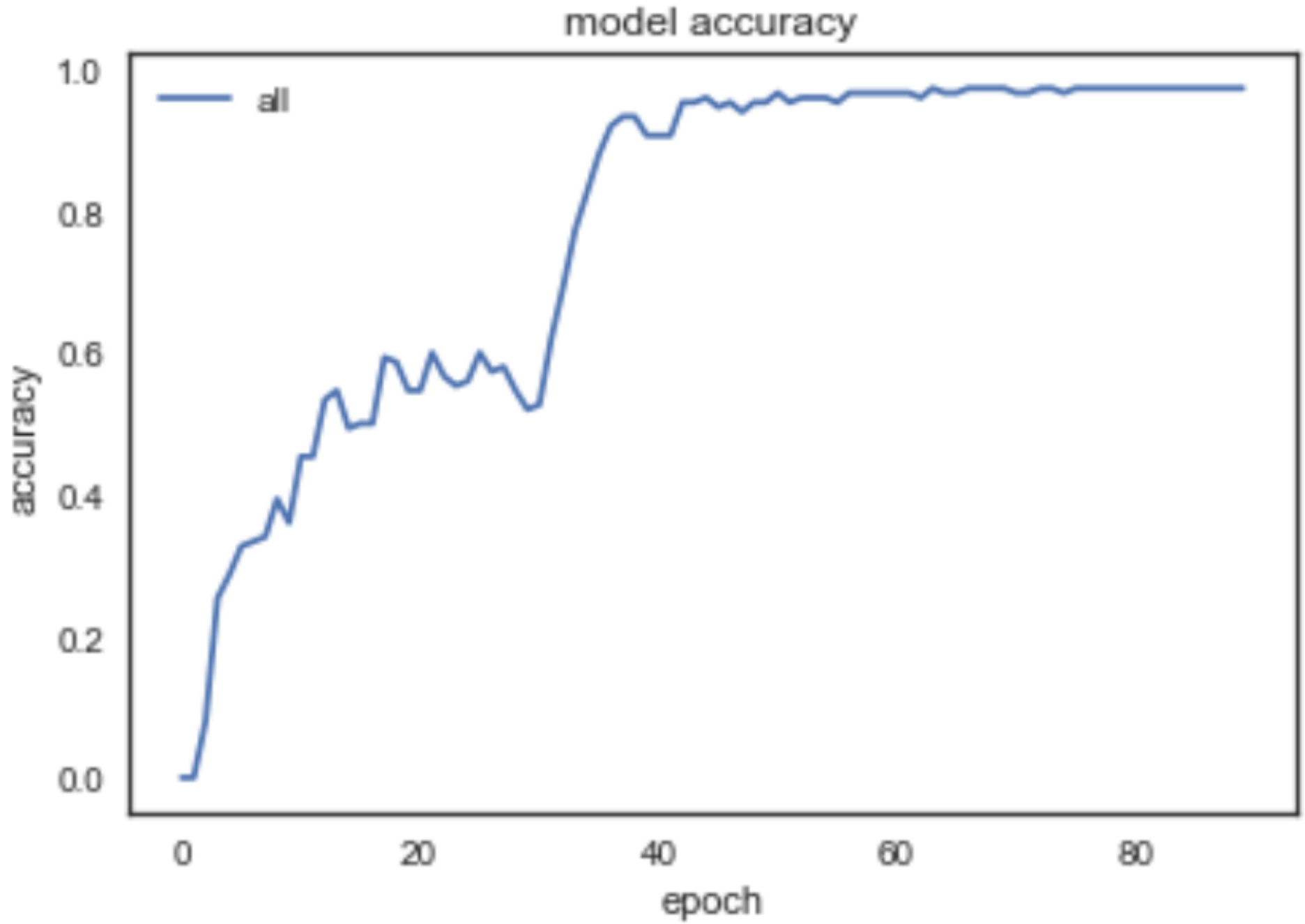
```
Epoch 1/90
150/150 [==============================] - 0s 1ms/step - loss: 2.2679 - acc: 0.0000e+00
Epoch 2/90
150/150 [==============================] - 0s 136us/step - loss: 1.9836 - acc: 0.0000e+00
Epoch 3/90
150/150 [==============================] - 0s 146us/step - loss: 1.7694 - acc: 0.0800
Epoch 4/90
150/150 [==============================] - 0s 136us/step - loss: 1.6021 - acc: 0.2533
Epoch 5/90
150/150 [==============================] - 0s 136us/step - loss: 1.4733 - acc: 0.2867
Epoch 6/90
150/150 [==============================] - 0s 123us/step - loss: 1.3785 - acc: 0.3267
Epoch 7/90
150/150 [==============================] - 0s 146us/step - loss: 1.3026 - acc: 0.3333
Epoch 8/90
150/150 [==============================] - 0s 146us/step - loss: 1.2400 - acc: 0.3400
Epoch 9/90
150/150 [==============================] - 0s 123us/step - loss: 1.2000 - acc: 0.3933
```

Epochs = 做 90 次疊代 (iteration)
Batch_size = 每次處理 5 筆資料

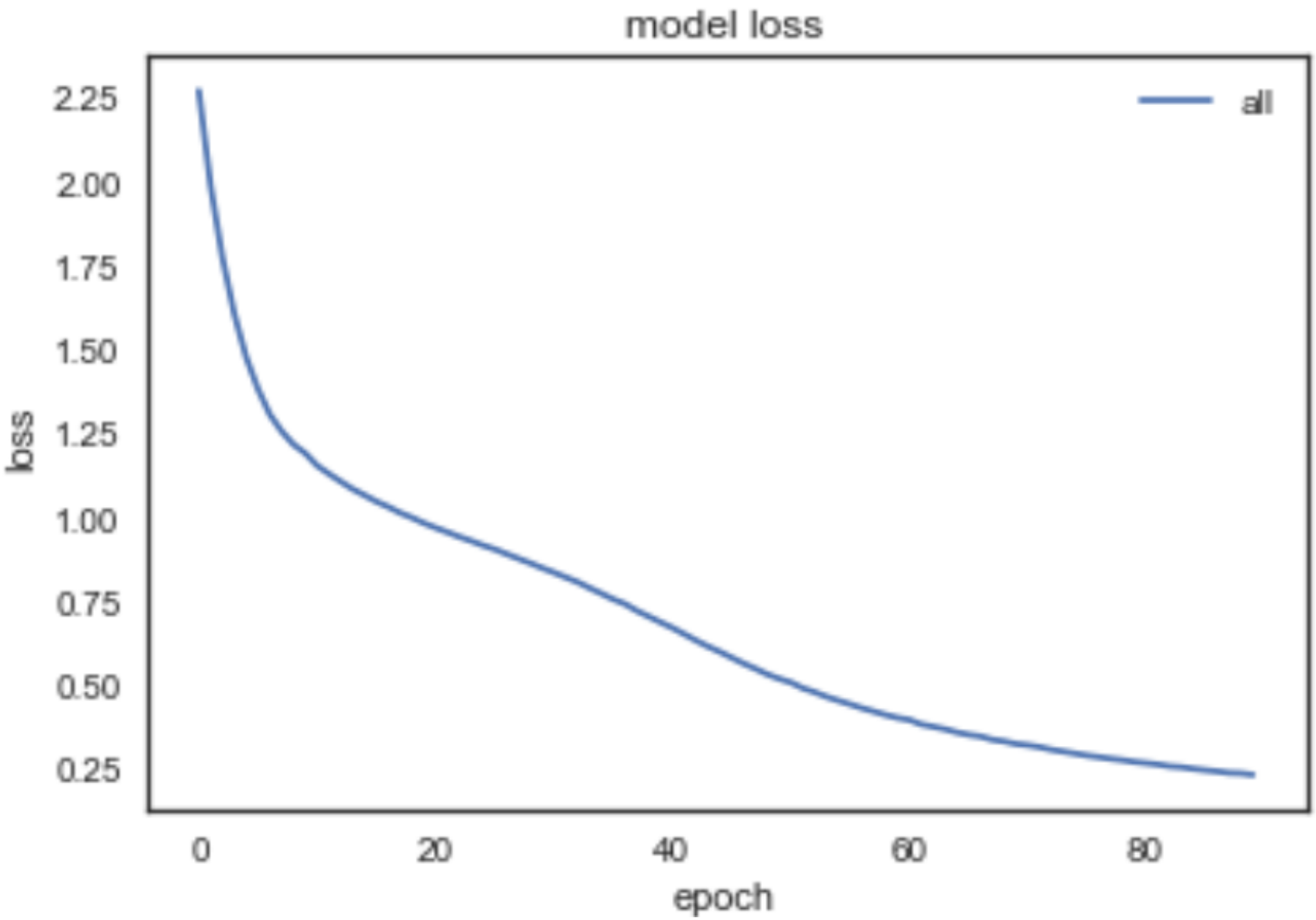
```
Epoch 82/90
150/150 [==============================] - 0s 130us/step - loss: 0.2601 - acc: 0.9733
Epoch 83/90
150/150 [==============================] - 0s 130us/step - loss: 0.2545 - acc: 0.9733
Epoch 84/90

150/150 [==============================] - 0s 123us/step - loss: 0.2521 - acc: 0.9733
Epoch 85/90
150/150 [==============================] - 0s 130us/step - loss: 0.2470 - acc: 0.9733
Epoch 86/90
150/150 [==============================] - 0s 136us/step - loss: 0.2431 - acc: 0.9733
Epoch 87/90
150/150 [==============================] - 0s 140us/step - loss: 0.2395 - acc: 0.9733
Epoch 88/90
150/150 [==============================] - 0s 143us/step - loss: 0.2349 - acc: 0.9733
Epoch 89/90
150/150 [==============================] - 0s 140us/step - loss: 0.2337 - acc: 0.9733
Epoch 90/90
150/150 [==============================] - 0s 136us/step - loss: 0.2293 - acc: 0.9733
```

```
# summarize history for accuracy
plt.plot(history.history['acc'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['all'], loc='upper left')
plt.savefig('acc.png')
plt.show()
```


model accuracy
all
accuracy
epoch
1.0
0.8
0.6
0.4
0.2
0.0
0
20
40
60
80

```
# summarize history for loss
plt.plot(history.history['loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['all'], loc='upper right')
plt.savefig('loss.png')
plt.show()
```


model loss
all
2.25
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
loss
0
20
40
60
80
epoch

```
scores = model.evaluate(X,answer_Y)
```

```
150/150 [==============================] - 0s 227us/step
```

```
print("Accuracy:%.2f%%"%(scores[1]*100))
```

```
Accuracy:97.33%
```

Try the followings:

(1) Use different seed number
(2) Use only 3 features as inputs
(3) Use Y_onehot in model.fit
(4) Use loss= 'mse’
(5) Use encoded_Y and modify the output layer

Thank You